

## Material complementario - Módulos ES6



¡Hola! Te damos la bienvenida a esta nueva guía de material complementario.

### ¿En qué consiste esta guía?

La siguiente guía de estudio tiene como objetivo practicar y ejercitar los contenidos que hemos visto en clase, además de profundizar temas adicionales que complementan aquellos que revisamos.

Dentro de los cuales se encuentran:

- Conocer las diferencias entre CommonJS y ES6 Modules.

**¡Vamos con todo!**



## Tabla de contenidos

|                                                        |          |
|--------------------------------------------------------|----------|
| <b>Recordando lo aprendido</b>                         | <b>2</b> |
| Requiere                                               | 2        |
| Import                                                 | 3        |
| Módulos ES6                                            | 3        |
| <b>¡Manos a la obra! - Módulos con import y export</b> | <b>3</b> |
| fs.writeFile                                           | 5        |
| Promise                                                | 5        |
| <b>¡Manos a la obra! - todos con Node JS</b>           | <b>6</b> |



**¡Comencemos!**

## Recordando lo aprendido

Cómo vimos en la guía de estudio, node.js viene con una variada galería de módulos que podemos utilizar en nuestros proyectos.

Uno de sus más populares es el módulo File System. Este módulo nos permite interactuar con el sistema de archivos de nuestra computadora.

Ejemplo:

```
const fs = require("fs");

fs.writeFileSync("hello.txt", "Hello from Node.js 😊");

console.log("archivo creado");
```

## Requiere

La palabra clave `require` es una característica propia de CommonJS, un sistema de módulos utilizado en Node.js para importar módulos y hacerlos disponibles en una aplicación.

## Import

`import` es la palabra clave utilizada en ECMAScript 6 (también conocido como ES6 o ES2015) para importar módulos. A diferencia de `require`, `import` es una característica de JavaScript y no es compatible con CommonJS por defecto.

## Módulos ES6

Al igual como trabajamos en la unidad de React.js, los módulos ES6 son una forma de organizar el código en archivos separados.

- Cada archivo es un módulo y puede exportar código utilizando la palabra clave `export`.
- Otros módulos pueden importar ese código utilizando la palabra clave `import`.

La diferencia es que en node.js no viene por defecto, pero su activación es muy sencilla. Veamos entonces un ejemplo de implementación.



## ¡Manos a la obra! - Módulos con import y export

- **Paso 1:** En tu terminal, crea un nuevo directorio para tu proyecto. Puedes hacerlo con el siguiente comando:

```
mkdir mi_proyecto  
cd mi_proyecto
```



Es importante señalar que tienes que estar en el directorio de tu proyecto para poder seguir trabajando.

- **Paso 2:** Luego, inicializa un proyecto de node.js con el siguiente comando:

```
npm init -y
```

Esto creará un archivo package.json en tu directorio. Este archivo contiene información sobre tu proyecto y sus dependencias.



En la siguiente clase se entrará en más detalles e instalaremos módulos externos vía npm.

- **Paso 3:** Configurar package.json para módulos ES6.

Después de completar npm init, tu proyecto tendrá un archivo package.json. Abre este archivo en un editor de texto y agrega la siguiente línea.

```
"type": "module"
```

Asegúrate de que tu archivo package.json se vea así:

```
{  
  "name": "mi_proyecto",  
  "version": "1.0.0",  
  "description": "Mi proyecto Node.js",  
  "main": "index.js",  
  "type": "module", // Agrega esta línea  
  "scripts": {
```

```
"test": "echo \"Error: no test specified\" && exit 1"
},
"author": "Tu Nombre",
"license": "MIT"
}
```

Este archivo le dice a Node.js que tu proyecto utilizará módulos ES6. No necesitas estar creando este archivo manualmente, ya que al ejecutar el comando `npm init -y` se crea automáticamente.

- **Paso 4:** Veamos un ejemplo de implementación del módulo `fs` con ES6. Ya tenemos nuestro proyecto configurado para trabajar con módulos ES6, podemos crear un archivo `index.js` y escribir el siguiente código:

```
import fs from "fs";

fs.writeFileSync("hello.txt", "Hello from Node.js 😊");

console.log("archivo creado");
```



**¿Esto te resulta familiar?** En la clase vimos un código muy parecido, en este caso la diferencia es que la importación la estamos realizando módulos ES6 (Sintaxis de `import` y `export`)

## fs.writeFile

A diferencia de `fs.writeFileSync`, que es una función sincrónica y bloqueante, `fs.writeFile` es una función asíncrona que no bloquea el hilo principal de Node.js mientras se realiza la escritura en el archivo.

`fs.writeFile` acepta una función de devolución de llamada (callback) que se ejecuta una vez que se ha completado la escritura del archivo.

```
import fs from "fs";

// fs.writeFileSync("hello.txt", "Hello from Node.js 😊");

fs.writeFile("hello.txt", "Hello from Node.js 😊", (err) => {
  if (err) {
    console.log(err);
  } else {
```

```
    console.log("archivo creado");  
  }  
});
```

## Promise

También se puede utilizar fs.writeFile con promesas. Para ello, se debe utilizar la versión de fs.writeFile que devuelve una promesa.

```
import { writeFile } from "node:fs/promises";  
  
try {  
  await writeFile("hello.txt", "Hello from Node.js 😊");  
  console.log("archivo creado");  
} catch (error) {  
  console.log(error);  
}
```



## ¡Manos a la obra! - toDos con Node JS

A continuación, realizaremos un ejercicio con Node JS que consiste en una aplicación para crear tareas. Esta aplicación será ejecutada mediante la terminal o consola de comandos y permitirá agregar y listar tareas.

- **Paso 1:** En la misma aplicación del ejercicio anterior, creamos un nuevo archivo llamado toDos.js
- **Paso 2:** Realizamos la importación de los módulos requeridos para el ejercicio

```
import { readFile, writeFile } from "node:fs/promises";
```

- **Paso 3:** Ahora, definimos las constantes que nos permitirán capturar los datos que sean ingresados mediante terminal.

```
const arg = process.argv.slice(2);  
const [option, text] = arg;
```

- **Paso 4:** Luego, creamos el conjunto de funciones que nos permitirán leer, escribir, agregar y listar las tareas.

```
const readTasks = async () => {
  try {
    return JSON.parse(await readFile("tasks.json", "utf-8"));
  } catch (error) {
    await writeFile("tasks.json", JSON.stringify([]));
    return [];
  }
};

const writeTasks = async (tasks) => {
  try {
    await writeFile("tasks.json", JSON.stringify(tasks));
  } catch (error) {
    console.log(error);
  }
};

const addTask = async (text) => {
  const tasks = await readTasks();
  tasks.push({ text, completed: false });
  await writeTasks(tasks);
  console.log("task added");
};

const listTasks = async () => {
  const tasks = await readTasks();
  console.log(tasks);
};

const main = async () => {
  if (option === "add") {
    await addTask(text);
  }

  if (option === "list") {
    await listTasks();
  }
};

main();
```

Ahora, ejecutemos el programa `todos.js` con el siguiente comando `node index.js add "Estudiar módulos ES6"`

Al ejecutar este comando en terminal, se va a generar un archivo llamado `tasks.json` con la siguiente información:

```
[  
  { "text": "Estudiar módulos ES6", "completed": false }  
]
```

Ahora, si ejecutamos el comando `node index.js list` veremos en la terminal la tarea creada.

```
PS D:\Node JS> node .\todos.js list  
[ { text: 'Estudiar módulos ES6', completed: false } ]
```

Imagen 1. Tareas listadas en consola  
Fuente: Desafío Latam



En el caso de la función `main`, podemos mejorarla haciendo uso del `switch` en JavaScript, veamos un ejemplo de cómo implementar la validación con `switch`.

Función autoinvocada `main` con `switch`.

```
(async function main() {  
  switch (option) {  
    case "add":  
      await addTask(text);  
      break;  
    case "list":  
      await listTasks();  
      break;  
    default:  
      console.log("Unknown option:", option);  
  }  
})();
```

Hasta este punto, tenemos una aplicación funcional de tareas con Node JS. Anímate e intenta agregar más opciones y funcionalidades al programa, como actualizar, eliminar tareas y modularizar el código (las funciones) utilizando `import` y `export`.